Rock-E User Manual

Rock-E has a number of autonomous and remote control functionalities that will be listed in this document. Listed below are the different sections and operating modes that will be discussed.

General Information **2**

24 and 12 Volt Batteries Connections 2

Necessary Laptop Connections 3

Arduino Due's and micro-ROS Agent Connection 4

Subscriber Arduino Due and Optional Peripherals 7

IMU Calibration 7

Colcon Build 7

.bashrc Script 8

/udev Rules 8

"alias" Commands 9

SLAM "transform_publish_period" Parameter 10

Sensor Connections and Power 11

Remote Control Mode **12**

Necessary Laptop Connections 12

Setup and Use 12

Full Autonomous Mode **14**

Necessary Laptop Connections 14

Setup and Use 14

Autonomous Mode without GPS **17**

Necessary Laptop Connections 17

Setup and Use 17

How to Generate and Save a Map **19**

Necessary Laptop Connections 19

Setup and Use 19

Map Navigation Mode **22**

Necessary Laptop Connections 22

Setup and Use 22

General Information

Depending on the operating mode, a number of things should be considered:

24 and 12 Volt Batteries Connections

In order for the robot to perform as expected, it obviously has to be powered properly. The 24 Volt Battery provides power to the motor drivers. Generally, when the robot is being stored, the connection seen in **Figure 1** below is unplugged. Make sure this is plugged in before operating (See **Figure 2**). Additionally, the 12 Volt Battery needs to be properly connected. Generally, the 12 Volt Battery is disconnected at the front as seen in **Figure 3**. Make sure to connect the red wire to the positive terminal (**Figure 4**). In addition to this, there is a 12 Volt switch at the rear and on the left of the vehicle as seen in **Figure 5**. When this switch is down, the 12 Volt battery is not providing power. Make sure to keep this switch off as often as possible to conserve the 12 Volt Battery which powers the fan, safety light, Encoders, and Ultrasonic Sensors. It is also inline with the E-Stops so the vehicle will not move without this on. The E-Stops also need to be released in order for the vehicle to move (**Figure 6**).

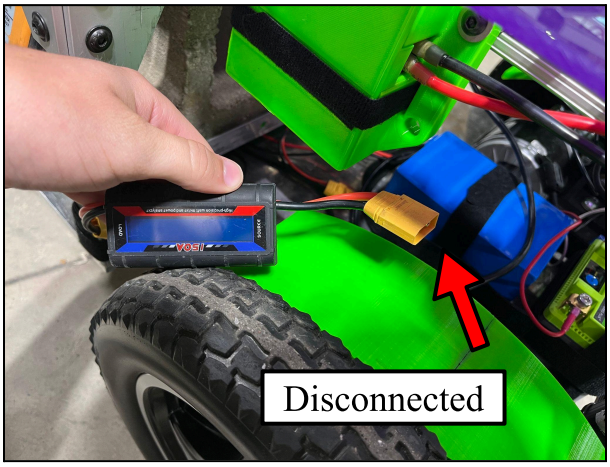


Figure 1: Disconnected 24 Volt Battery

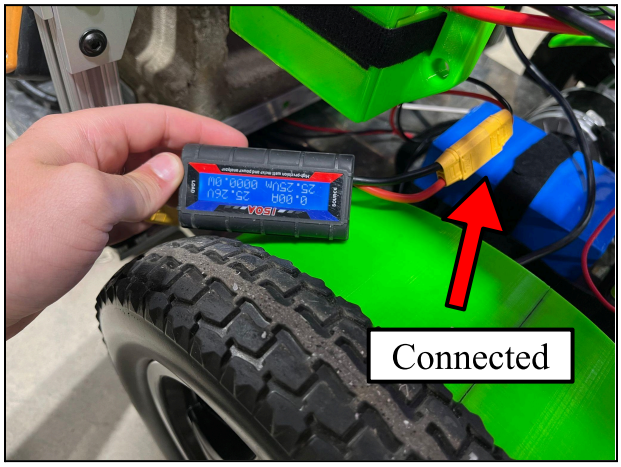


Figure 2: Connected 24 Volt Battery

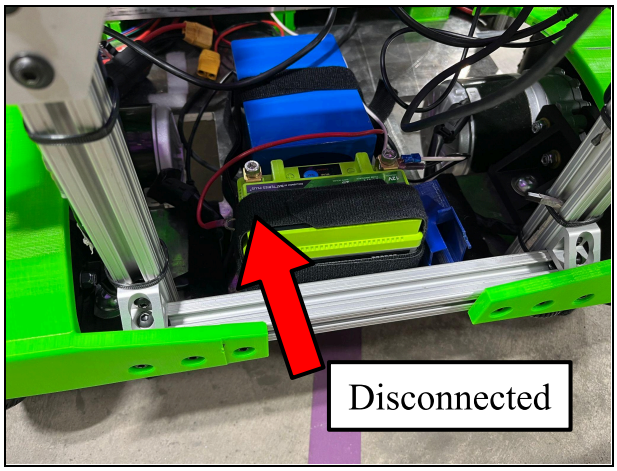


Figure 3: Disconnected 12 Volt Battery

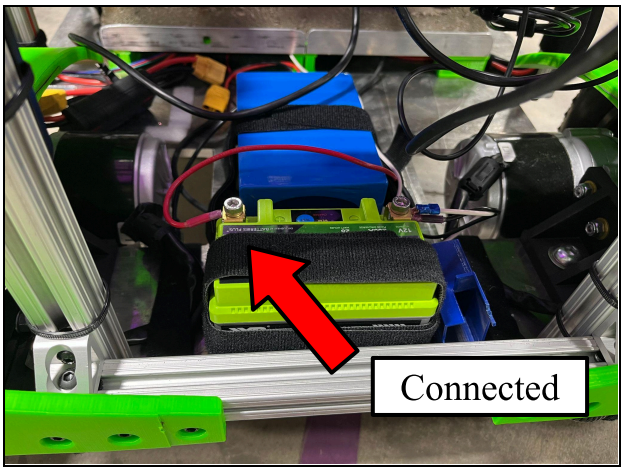


Figure 4: Connected 12 Volt Battery



Figure 5: 12 Volt Power Switch

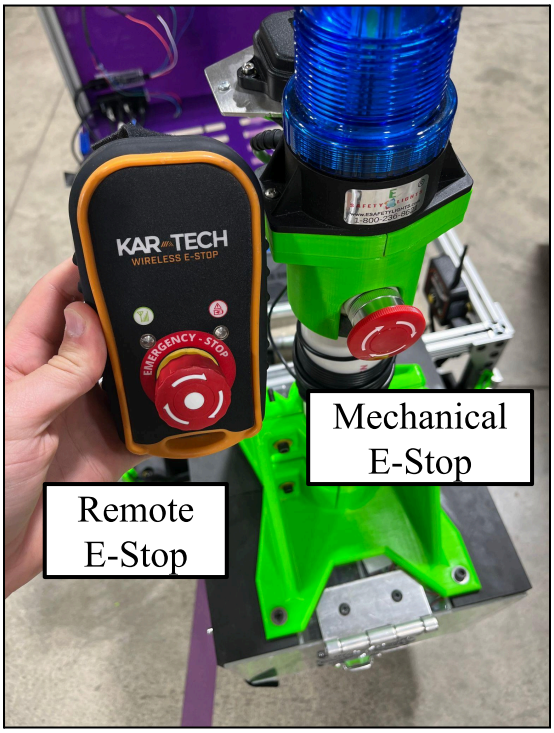


Figure 6: E-Stop Buttons

Necessary Laptop Connections

Each mode has a section detailing the Necessary Laptop Connections. A lot of the sensor drivers will return errors on execution if the proper sensors are not plugged in and detected by the computer. In addition to this, each operating mode requires information from the sensors listed. Make sure to connect each sensor so that no errors are encountered and the proper information is being sent to the computer

Arduino Due's and micro-ROS Agent Connection

In addition to the main laptop, two microcontrollers are used. These devices do a number of things, but the most important function is to relay data back and forth between the main computer. The Publisher Arduino Due should be connected at all times in most modes while the Subscriber Arduino Due is optional (See next section). **Figure 7** shows each microcontroller as located in the electronics box.

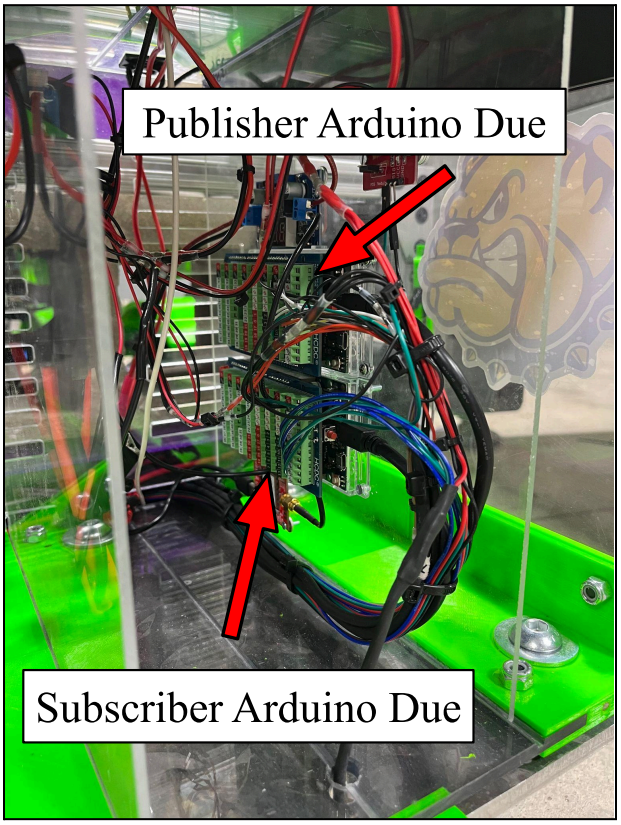


Figure 7: Microcontroller Placement

In order to properly connect these with the micro-ROS agent, follow these instructions:

In a terminal, type the "pub" or "sub" command depending on which Arduino Due you are trying to connect. The "pub" command is used to run the micro-ROS agent in order to establish communication between the Publisher Arduino Due and the computer. The Publisher Arduino Due has connections to both Encoders and both Motor Drivers and is the upper microcontroller located in the electronics box. The "sub" command does the same for the Subscriber Arduino Due which is connected to the three Ultrasonic Sensors and the Safety Light. After running either of these commands, 3 things can occur:

1. The following will occur if the Arduino is not correctly connected to the USB port of the computer (**Figure 8**). If this is shown, make sure you are plugged into the right Arduino Due and that the USB connection is secure.

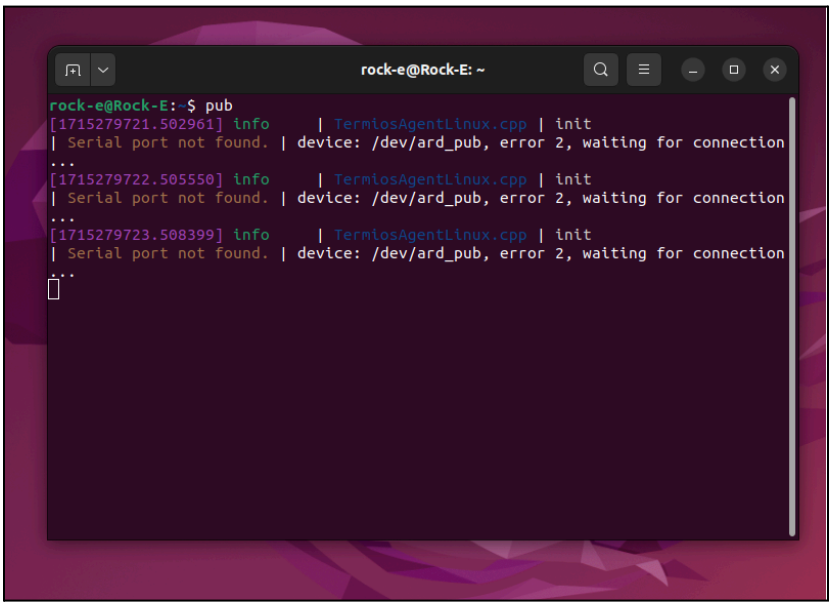


Figure 8: Terminal Output of Bad USB Connection

2. **Figure 9** shows the terminal output if the Arduino is properly connected to the computer by USB, but if it has not yet established communication with the ROS2 software. If you wait a couple of seconds and nothing else occurs, click the reset button on the Arduino Due you are trying to connect. If this does not work, restart the terminal.

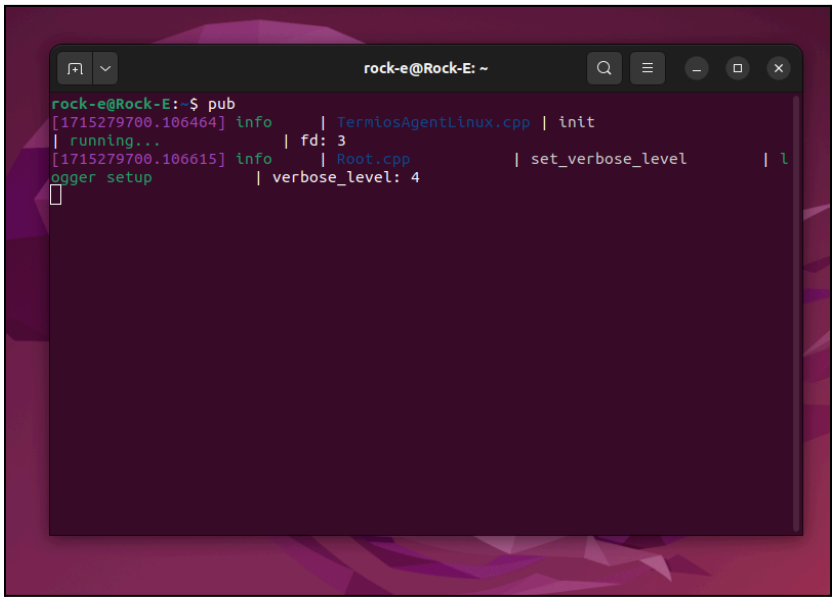


Figure 9: Terminal Output Before Established Communication

3. The following will appear when the Arduino is properly connected by USB and if it has established communication with the ROS2 software (**Figure 10**). When you see this, the Arduino Due has been properly connected and the peripherals attached to that respective microcontroller should be functional.

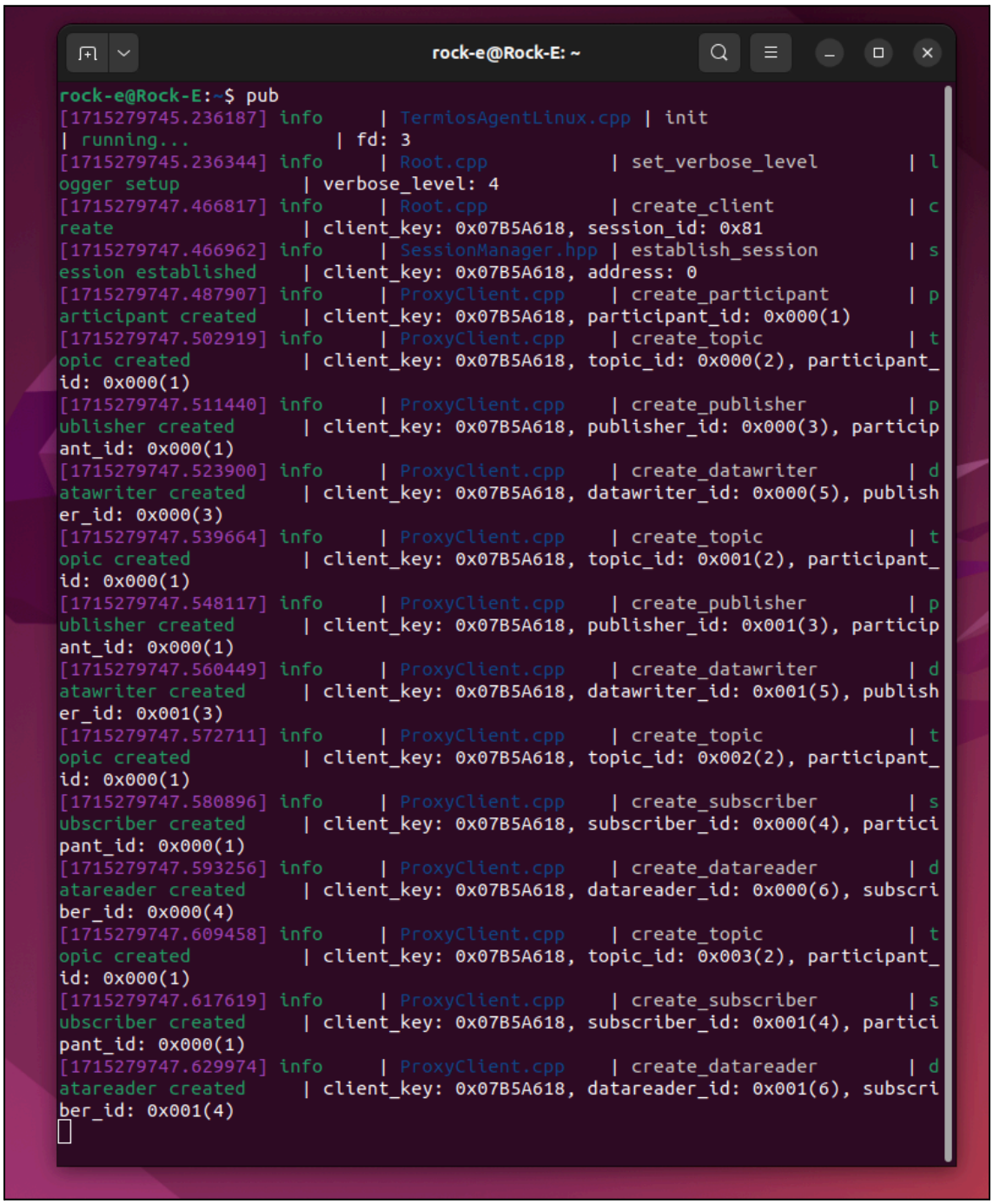


Figure 10: Terminal Output After Established Communication

Subscriber Arduino Due and Optional Peripherals

In every mode, the Safety Light and Ultrasonic Sensors are optional for operation. Because of this, these components are connected to the Subscriber Arduino Due, which is the lower microcontroller found in the electronics box (**Figure 7**). If you wish to have these components operating, connect the Subscriber Arduino Due to the laptop, run the "sub" command, and follow the steps in the previous section.

IMU Calibration

Another thing that is optional for operation is calibrating the Adafruit BNO055 IMU sensor. Still it is **highly** recommended to do this each time for increased accuracy and performance of the system. To calibrate the IMU, use the steps found at this link. To track the progress of the calibration, use the "calib" command in a terminal. This will print the system (sys), gyrometer (gyr), accelerometer (acc), and magnetometer (mag) sensors' calibration status. When all of these calibration statuses read 3, the IMU is properly calibrated (**Figures 11 and 12**).

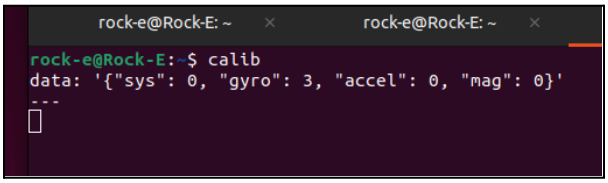


Figure 11: Output of Uncalibrated IMU

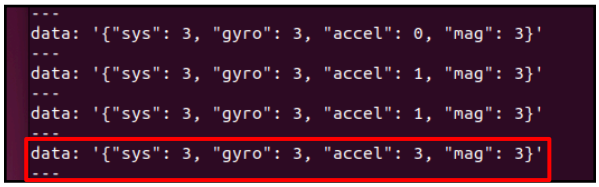


Figure 12: Output of Calibrated IMU

Colcon Build

Every time code is modified, the system needs to be rebuilt. This is done using the "colcon build" or "cb" command, but it is **NECESSARY** to do so in the "/ros2_ws" directory. If not, the system could run into a lot of errors that are hard to fix. Make sure to go to the proper directory first by using "cd ros2_ws" and then rebuild the workspace using the "cb" command. Try to avoid doing this unless you are modifying the SLAM "transform_publish_period" Parameter or the waypoints for Full Autonomous Mode. See **Figure 13**.

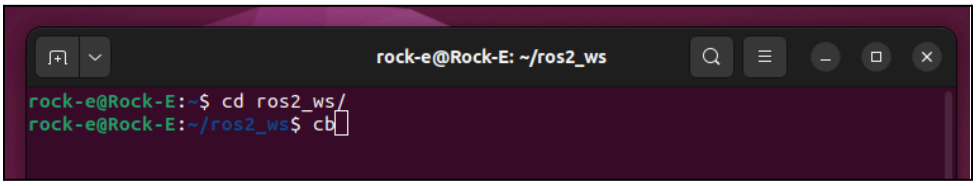


Figure 13: Colcon Build Process

.bashrc Script

The .bashrc script is a file that holds a number of necessary and custom code that is executed each time a terminal is opened. This script has a number of functions, as it is used to source a number of things, set a common baud rate, and to assign the ZED library (can be accessed by using “gedit ~/.bashrc”). Sourcing, put simply, is used to let the terminal know what directory you desire to access. Since the computer is mainly used to operate the robot, the ROS2 Workspace (/ros2_ws) is sourced automatically each time along with a number of other things. It is also used to save the “alias” commands detailed in a different section (See alias Commands).

/udev Rules

With the Linux operating system, a number of things are made more difficult. To customize this system for convenience, /udev rules can be used. On the school computer a number of /udev rules have been added. For example, the Linux operating system, due to its bare-bones structure, requires root permission to give read-write access to USB ports. What this means is that when you plug in an unknown peripheral into the laptop, it will not be able to relay its information to and from the laptop without first getting permission. This can be done in a terminal, but is only temporary for as long as that terminal is open. To combat this, a /udev rule was written to allow all USB devices read-write access no matter what for convenience. In addition to this, a /udev rule was written to assign static USB ports for each peripheral being used. This is necessary as each time something is plugged into the laptop, it assigns the USB device a name based on when it was plugged in. Because each sensor driver requires a device path to function, this can become tricky as more and more sensors are being plugged into the computer. As a workaround, a rule was written to assign the same name to a peripheral device each time it is plugged in to avoid conflict. For example, the GPS, when plugged in first, will receive the name “/dev/ttyACM0” without the previously described /udev rule. If it is plugged in after something else, it will receive the name “/dev/ttyACM1” without a static usb port. You can see where this could lead to problems, so a /udev rule was written to name the GPS “/dev/gps” each time it is plugged in. This way, the sensor driver can be pathed to this sensor each time without fail (these rules can be accessed and changed using “sudo nano /etc/udev/rules.d/50-ds4drv.rules”). If the computer is wiped, these will have to be rewritten.

“alias” Commands

Due to the length and large number of terminal scripts that can be used, a number of “alias” commands were developed for the school computer. These commands are written and found in the .bashrc script (accessed by using “gedit ~/.bashrc”). These custom commands greatly shorten the long scripts that are needed to execute certain programs. Listed below in **Table 1** are a number of “alias” commands that will be mentioned in this tutorial.

alias Command	Corresponding Terminal Command
pub	ros2 run micro_ros_agent micro_ros_agent serial –dev/dev/ard_pub
rc	ros2 launch rock-e rc.launch.py
cost	ros2 launch rock-e cost.launch.py
sub	ros2 run micro_ros_agent micro_ros_agent serial –dev/dev/ard_sub
global	ros2 launch robot_localization dual_ekf_navsat_example.launch.py
local	ros2 launch robot_localization ekf.launch.py
slam	ros2 launch slam_toolbox online_async_launch.py
nav2	ros2 launch nav2_bringup navigation_launch.py
log	ros2 run nav2_gps_waypoint_follower_demo logged_waypoint_follower
calib	ros2 topic echo /bno055/calib_status
rviz2	ros2 run rviz2 rviz2
cmd	ros2 topic echo /cmd_vel
amcl	ros2 launch rock-e amcl.launch.py
ms	Types: “ros2 run nav2_map_server map_saver_cli -f ”
ml	Types: “ros2 launch nav2_bringup localization_launch.py map:= ”

Table 1: “alias” Commands

SLAM "transform_publish_period" Parameter

Depending on the mode, a certain parameter in the "slam_toolbox" package needs to be set respectively. To access these parameters, open Visual Studio Code located in the Favorites Tab. Navigate to "ros2_ws/src/slam_toolbox/config/mapper_params_online_async.yaml" where the parameters are located. The "transform_publish_period" parameter determines whether or not the SLAM software will provide the map -> odom transform. In Autonomous Mode without GPS and when generating a map (See How to Generate and Save a Map), the map frame needs to be generated by SLAM, so the parameter should be set to a non-zero value (usually 0.02). In Full Autonomous Mode, the dual Extended Kalman Filter is providing the map -> odom transform, so this parameter should be set to zero (needs to be 0.0 to fit "double" variable type). After this is set properly for the mode of operation, make sure to rebuild the workspace (See Colcon Build). See **Figure 14**, **15**, and **16** below.

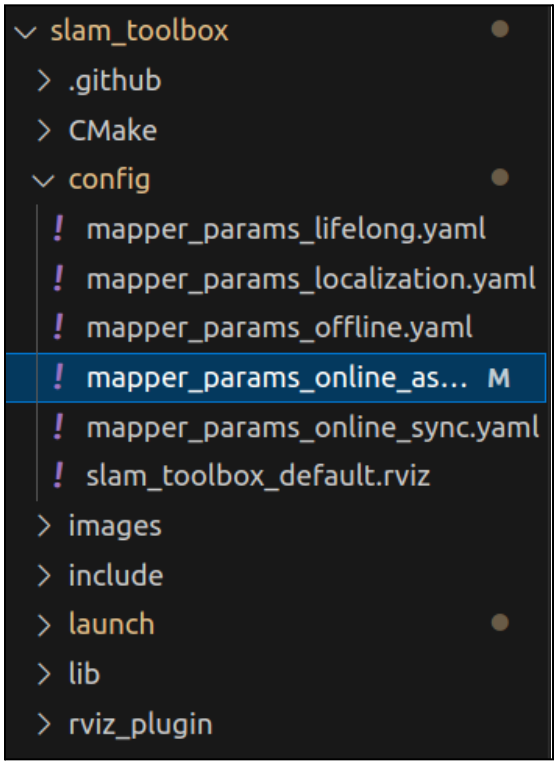


Figure 14: slam_toolbox Parameters Location

```
transform_publish_period: 0.02
```

Figure 15: Value for Most Modes

```
transform_publish_period: 0.00
```

Figure 16: Value for Full Autonomous Mode

Sensor Connections and Power

The school laptop has 2 USB-C and 2 USB Type A ports available for use. When running in Full Autonomous Mode, these ports are all used to connect the peripherals to the computer. The following is the typical setup for this mode. Usually, the computer is being powered by a Power Bank when it is running, which takes up one USB-C port. Additionally, the LiDAR power cable is generally connected to the Power Bank. It is highly recommended to use the Power Bank in order to avoid powering issues for the required components. The laptop should also be at full battery for the same reason, if possible. The other USB-C port is used to connect to the 4-port USB Hub. In the mentioned mode, the IMU, GPS, LiDAR (Data Cable), and Publisher Arduino Due are connected to this hub because of their importance. The Camera is then plugged into one of the USB Type A ports since it sometimes runs into errors when it is plugged into the hub. This is believed to occur due to a lack of power for the component. The final USB Type A port is then used for the optional Subscriber Arduino Due or for a wireless mouse connection. **Figure 17** below shows the setup for the left side of the laptop. The right side has the last Type A USB port.

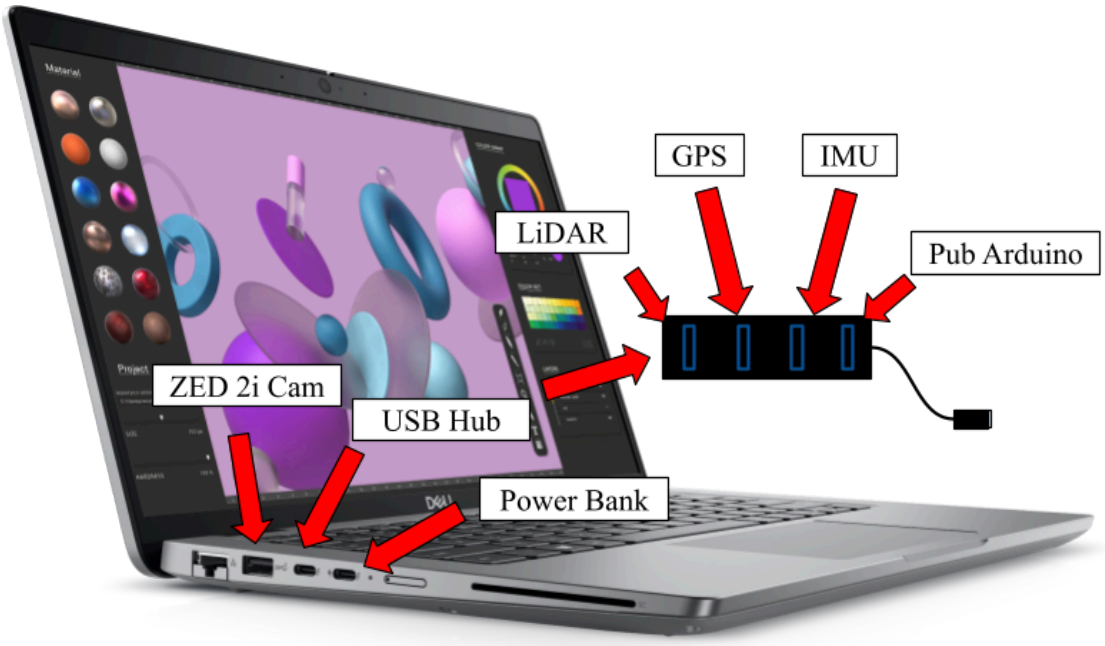


Figure 17: Typical Laptop Peripheral Setup

Remote Control Mode

This mode of operation is used to manually control the robot with a joystick controller. In this mode, a Bluetooth or USB wired controller is connected to the Laptop and is used to send velocity commands to the robot. Although almost any controller can be used to send velocity inputs to the robot, this system was configured with a PlayStation (4 or 5) controller. Other controllers like one for an Xbox can be used, but may not have the same button layout that will be described later (See Remote Control **Step 5**).

Necessary Laptop Connections

- Controller (Bluetooth or Wired)
- Publisher Arduino Due (Top)

Setup and Use

1. Go to the Bluetooth settings on the laptop. Connect to the wireless controller you plan to use with the system. You can also connect the controller with a USB cable if this is not working. The BlueTooth Settings are shown in **Figure 18** below.

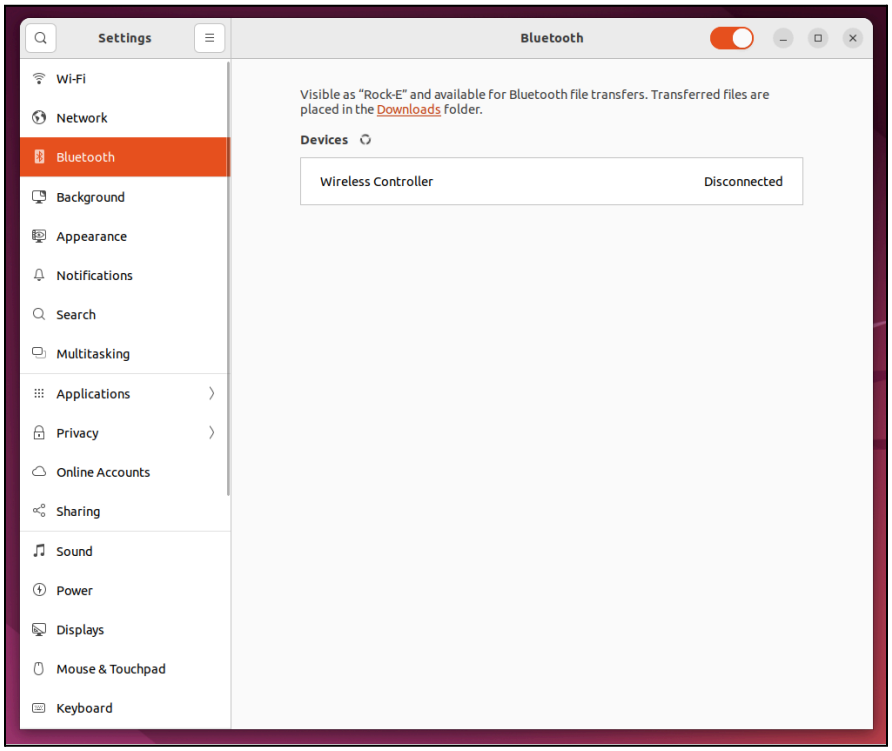


Figure 18: BlueTooth Settings

2. Open two terminals and type the commands “pub” in one and “rc” in the other.
3. Click “Enter” to run the “pub” command in the first terminal. This command runs the micro-ROS agent that allows the computer to communicate with the Arduino. (See Arduino Due’s Section)
4. After the Arduino has been properly connected, run the “rc” command in the other terminal. This command launches the nodes associated with controlling the motors in remote control mode.
5. The robot is now in remote control mode and is ready to publish velocity commands from the controller. Using the left joystick, you can control the movement of the robot. In order to do so, the “enable” button needs to be pressed and held or else no commands will be sent to the motors. In the current configuration the “L1” button (**Figure 19**) or left bumper is used to enable the velocity commands from the controller. When this is pressed and the left stick is moved, the robot will behave accordingly. There is also a “turbo” button that can allow the robot to travel at a higher speed. The current configuration has the “R1” button or right bumper as the “turbo” button.

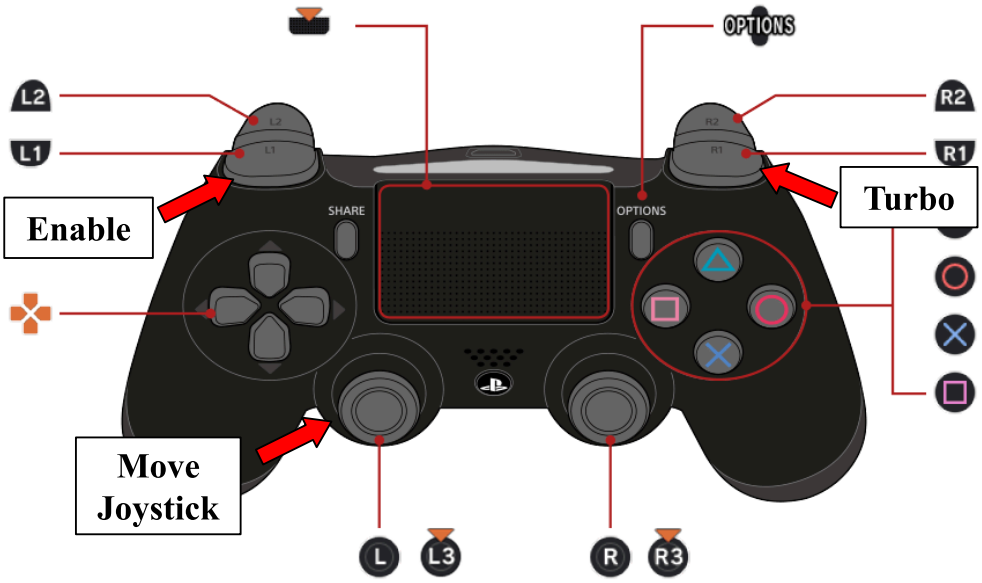


Figure 19: PlayStation 4 Controller Button Layout

Full Autonomous Mode

In full autonomous mode, Rock-E will navigate to GPS waypoints, avoid obstacles detected by the LiDAR, and stay between white lines. All sensors will be running simultaneously to provide the required data. Both the Camera and LiDAR data will be combined and used with SLAM to create the local and global costmaps used for navigation. The GPS, IMU, and encoder data will be combined in the local and global extended Kalman filters to provide the localization of the robot. Nav2 will also be running and using this data to perform the path planning and carry out motor commands. A minimum of 6 terminals are needed for this mode. There are 3 optional terminals that are usually run as well, but they are not necessary for this mode to function. These optional terminals are used to see the calibration values of the IMU during the calibration process, to run RViz2 to visualize the data, and to run the Subscriber Arduino Due.

Necessary Laptop Connections

- Camera
- LiDAR
- GPS
- IMU
- Publisher Arduino Due (Encoders and Motors)

Setup and Use

1. In the first terminal use the “pub” command to connect the Publisher Arduino to the computer. Make sure it is properly connected with the laptop (See Arduino Due’s).
2. In another terminal run the “cost” command to launch all of the sensor drivers, URDF, and other nodes. This launch is used to run a large number of nodes that can be easily reset simultaneously. This can run indefinitely and nothing will happen, but it will drain a lot of power as all the sensors will be active.
3. Usually after the execution of the previous step, the IMU is calibrated. As mentioned earlier, this is optional, but can affect the accuracy of the system. (See IMU Calibration)

4. In another terminal run the “global” command to launch the dual Extended Kalman Filter (EKF). Usually the 12 Volt switch on the back left of the robot is turned off to conserve battery power. However, make sure this is flipped on before running the “global” command so the encoders are powered and publish their respective data. Sometimes this launch will output an error saying *“Failed to meet update rate! Took 0.203820277999999993 62seconds. Try decreasing the rate, limiting sensor output frequency, or limiting the number of sensors.”* If this happens, stop the launch and rerun the command. Once it is successfully running, let the robot stay still for a minute to let the EKF process the GPS, IMU, and encoder data.
5. At this point, another optional step is usually taken. In another terminal, the “rviz2” command is run to start the RViz2 software. This is usually used to see how the output of the filter drifts (a result of inaccurate GPS Data) as time goes on. Make sure the global frame is set to “map” and that the RobotModel Display is on (**Figure 20**). The next step (6) is taken when the vehicle represented by the RobotModel display starts to converge to a 1 meter area. With the current setup, each grid represents a 1 meter square.

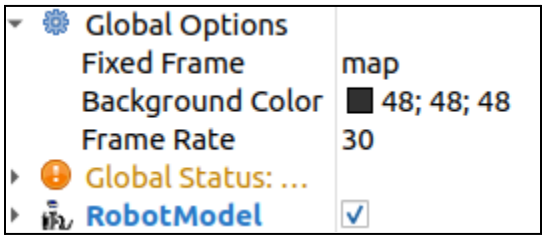


Figure 20: RViz2 Setup for Full Autonomous Mode

6. In another terminal, run the “nav2” command to launch a number of Nav2 nodes necessary for the path planning. Much like the “cost” launch, a lot of things are being run simultaneously, so this tends to drain the battery while it is active.
7. Following the start of Nav2, run the “slam” command in another terminal. This command runs the Simultaneous Localization and Mapping (SLAM) that is used to produce a map of the obstacles. Make sure that once this is run, no one passes in front of the LiDAR so that unwanted obstacles do not appear on the map. Additionally, make sure the code is configured properly for this mode (See SLAM “transfor_publish_period” Parameter)

8. At this point, the entire system is running and ready to be directed to a number of waypoints. To set the waypoints, open Visual Studio Code and navigate to "ros2_ws/src/navigation2_tutorials/nav2_gps_waypoint_follower_demo/config/demo_waypoints.yaml" which is shown in **Figure 21**. This file stores the coordinates and orientation of each waypoint (**Figure 22**). Make sure to give the correct latitude and longitude to at least 6 decimals and that the orientation is given in radians where East is 0 radians and North is $\frac{\pi}{2}$ or 1.5708 radians. After setting the desired coordinate and orientation in the .yaml file, make sure to use Colcon Build. Usually this is done before **Step 1** so that the system can build without everything else running. After this, run the "log" command in a **new** terminal window. This command has a built-in, 1-second delay along with executing the command phrase shown in "alias" Commands. This is because this script sends the desired waypoints to Nav2 to begin navigating. After executing this command, make sure to stand back and have the E-stop ready as the robot will begin navigating shortly.

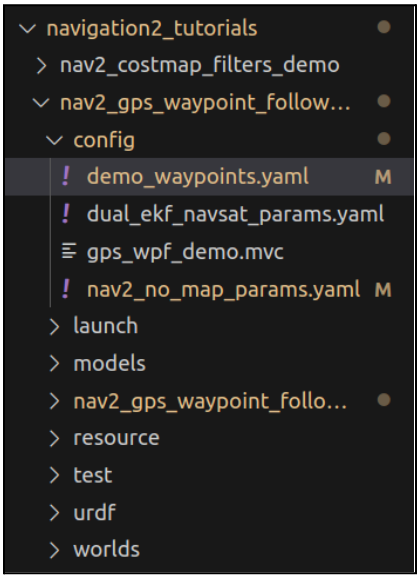


Figure 21: Waypoints .yaml File Location

```yaml
waypoints:
- latitude: 41.513412779741245
  longitude: -90.49104860061006
  yaw: 3.92699
- latitude: 41.5134082607519
  longitude: -90.49111967914442
  yaw: 3.1415
```

Figure 22: Example of Waypoints for Navigation

Autonomous Mode without GPS

In this mode, the robot has the ability to navigate to a waypoint while avoiding obstacles and staying between white lines; however, only one waypoint can be provided at a time, and the waypoint provided is not a GPS coordinate. Still, the setup is almost identical to the previous method of operation, and only the GPS sensor data is omitted from this mode.

Necessary Laptop Connections

- Camera
- LiDAR
- IMU
- Publisher Arduino Due (Encoders and Motors)

Setup and Use

1. Use the “pub” command to connect to the Publisher Arduino Due. (See Arduino Due’s)
2. Run the “amcl” command to start the required sensors. This starts all of the sensor drivers except for the GPS, so it doesn’t need to be plugged in.
3. Calibrate the IMU for increased accuracy. (See IMU Calibration)
4. In a new terminal, run the “local” command to start the Extended Kalman Filter for localization of the robot. Make sure to flip the 12 Volt Switch to power the encoders.
5. After, execute the “nav2” command in a terminal to launch the required nodes for Nav2.
6. This step and the next are interchangeable, just make sure not to pass in front of the LiDAR after running SLAM. Use the “slam” command to start SLAM. Make sure it is configured correctly by referring to SLAM “transfor_publish_period” Parameter.
7. In this mode, RViz2 is necessary as it is used to send the waypoint. Open the software using “rviz2” and configure RViz2 so that the global frame is “map”, the Map visual tool is subscribed to “/global_costmap/costmap”, and that the RobotModel is on. This way you can see the cost being generated around the detected obstacles and the localized robot. (**Figure 23**)

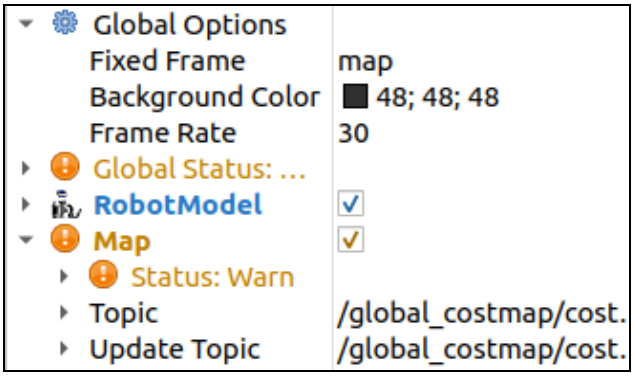


Figure 23: RViz2 Setup for Autonomous Mode without GPS

8. The system is now ready to be sent to a waypoint. At the top of RViz2, click on either the "2D Goal Pose" or "Nav2 Goal" Button (**Figure 24**). Then, click on the desired location and orientation you would like the robot to navigate to. Be careful after clicking and releasing the mouse to set the goal pose, as the robot will begin navigating (**Figure 25**).



Figure 24: 2D Goal Pose and Nav2 Goal Buttons

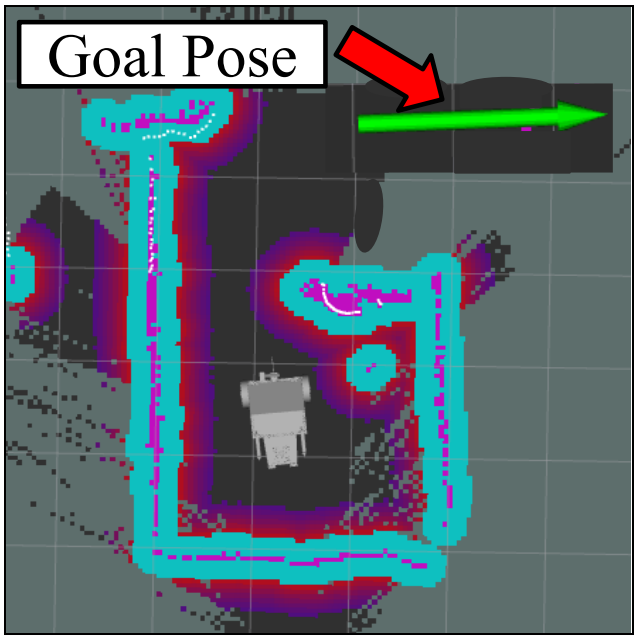


Figure 25: RViz2 with Costmap and Setting Goal Pose

How to Generate and Save a Map

Rock-E also has the ability to navigate through pre-built maps. In order to do so, the map needs to be made and saved beforehand. This can be done by putting the robot through the Full Autonomous Mode or Autonomous Mode without GPS and then saving the generated map, and it can be done in Remote Control Mode with the LiDAR and Camera active to detect obstacles for the map. This section details how to save a map with the vehicle operating by remote control, but if you wish to do so after the robot has navigated autonomously, simply follow **Step 7** to save the generated map.

Necessary Laptop Connections

The following connections are only necessary for mapping with remote control operation

- LiDAR
- Camera
- IMU
- Controller
- Publisher Arduino Due (Encoders and Motor)

Setup and Use

1. First set up the robot for Remote Control Mode.
2. Then, in a terminal, run the "amcl" command to start the sensors required for mapping.
3. At this time, the optional step of calibrating the IMU is taken. (See IMU Calibration)
4. In a new terminal, run the "local" command to start the Extended Kalman Filter which fuses the IMU and encoders data. If the 12 Volt Switch hasn't been flipped on, make sure to do so before executing this command.
5. Using the "slam" command in another terminal, start the mapping software for the environment. Make sure not to pass in front of the LiDAR after this step. With this running, the robot can be moved around the environment with the controller to generate a map of its surroundings. Before running this command, make sure to set the slam_toolbox "transform_publish_period" parameter accordingly. (See this Section)

6. Another optional step to open RViz2 is then executed, using the “rviz2” command. This is done in order to see the map being generated as Rock-E moves around the environment. The global frame should be set to “map” and the map visualization should be set to the “/map” topic in order to see the map being generated (**Figure 26**).

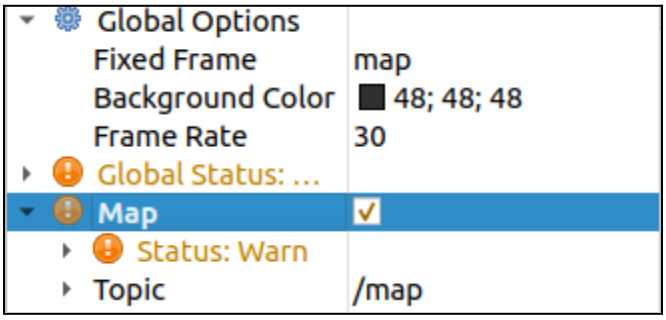


Figure 26: RViz2 Setup for Mapping

7. After the conclusion of Full Autonomous Mode, Autonomous Mode without GPS,or after you are satisfied with the manually generated map, the map is ready to be saved. Use the “ms” command in a new terminal. Unlike other commands, this only types out part of the required script to save a map because you need to provide a location to save the map and a unique name. Make sure to type out your desired save location and unique name after the script. Sometimes the “ms” command is finicky, so you can always press “enter” after the initial script has been typed, ignore the error, press the Up arrow key to retype the last command, and then type your save location and name before executing the command. An example of this can be seen in **Figure 27** below. You can double check that the map was saved successfully and look at the generated map by navigating to that same folder in the Files app found in the Favorites Tab (**Figure 28** & **29**). Usually, the maps are saved to the “ros2_ws/src/navigation2/nav2_bringup/maps” folder.

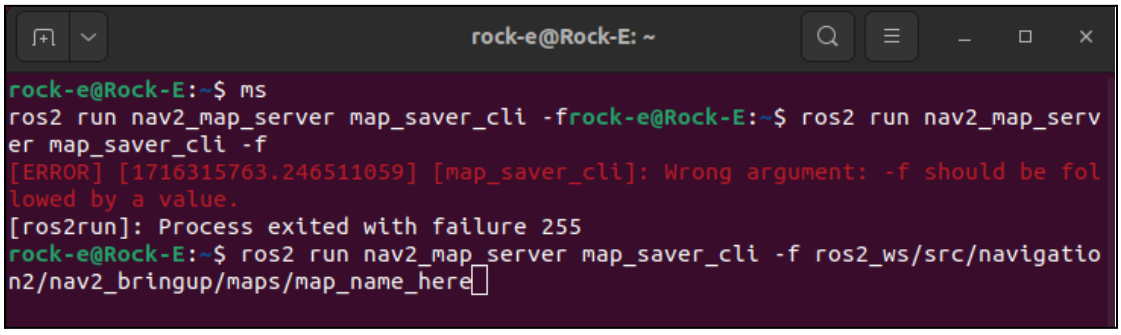


Figure 27: Map Saving Example

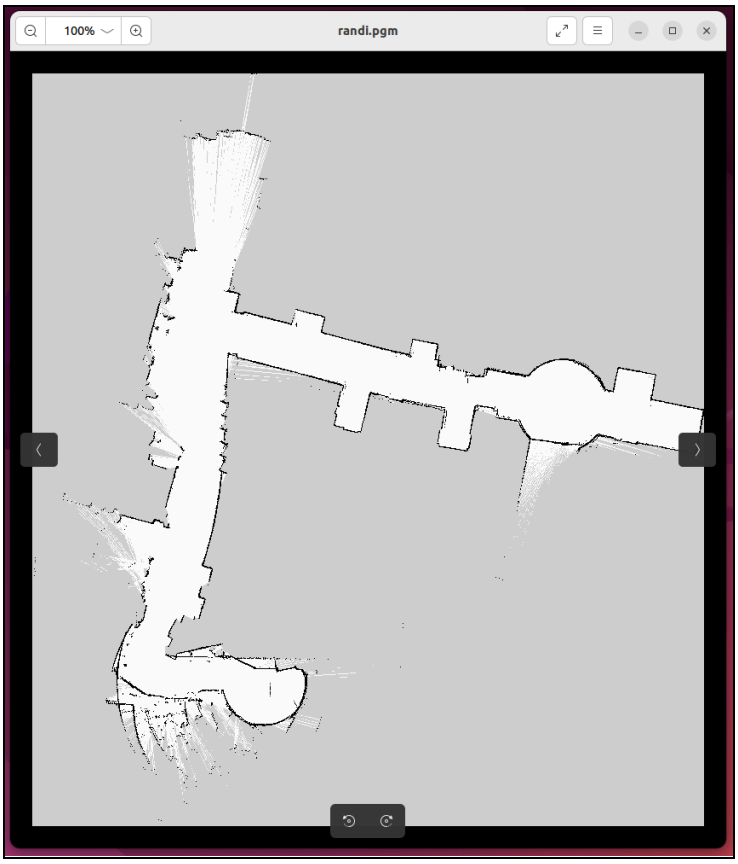


Figure 28: Example Map

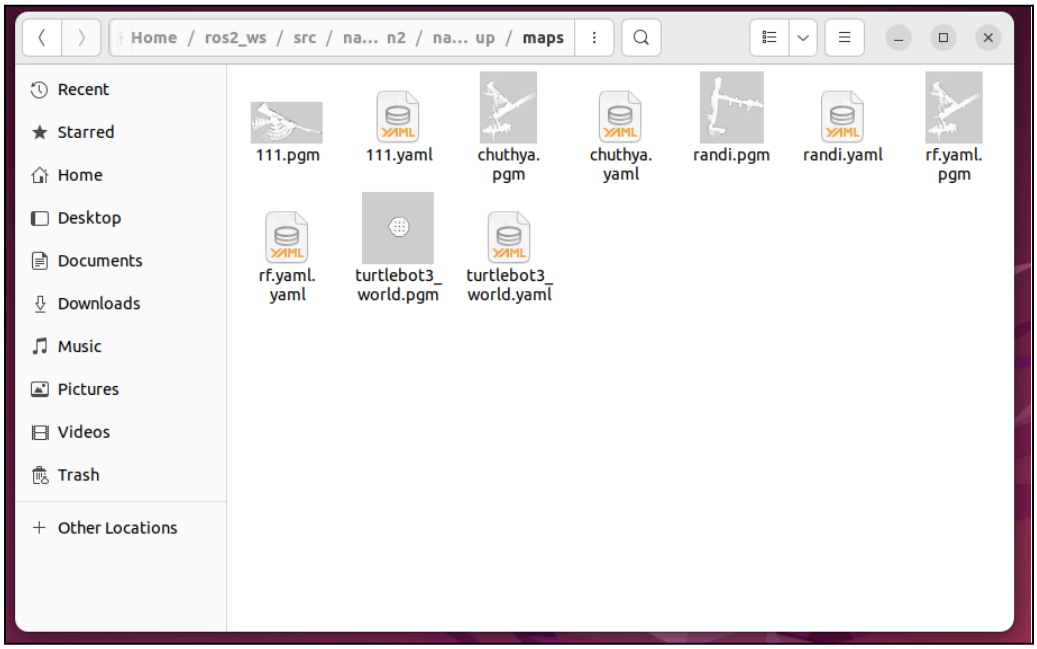


Figure 29: Map File Location

Map Navigation Mode

In this mode, the robot has the ability to navigate through an environment using a premade map. Much like the Full Autonomous Mode, a number of sensors are needed for the obstacle detection and localization. The Camera and LiDAR are used for obstacle detection and localization within the map, while the IMU and encoders are combined in a single Extended Kalman Filter to track the robots movements. Notice the exclusion of the GPS because it is not necessary for this mode. This makes it suitable for indoor areas that have been mapped prior to operation.

Necessary Laptop Connections

- Camera
- LiDAR
- IMU
- Publisher Arduino Due (Encoders and Motors)

Setup and Use

1. Using the “pub” command, connect the Publisher Arduino using micro-ROS (See Here).
2. In a new terminal, run the “amcl” command to run the sensors necessary for this mode.
3. Calibrate the IMU sensor (Optional, see IMU Calibration).
4. Then, execute the “local” command to localize the robot. Make sure the 12 Volt Switch is flipped on to power the wheel encoders
5. In most cases, RViz2 is optional; however, it must be used for Map Navigation Mode. Using the “rviz2” command, open the visualization tool. Make sure the global frame is set to “map” and the map visualization is subscribed to “/map”. See **Figure 30** below.

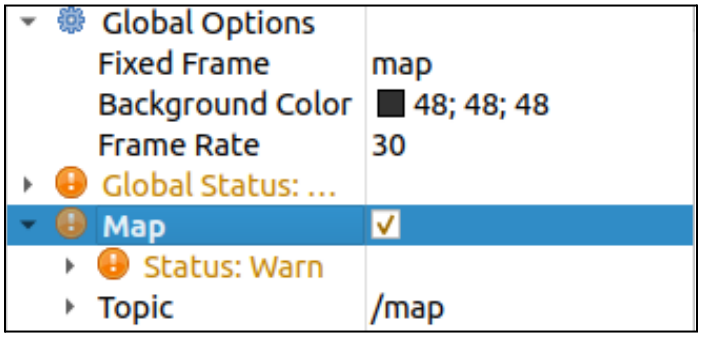


Figure 30: RViz2 Setup for Map Navigation Mode

6. Start the Nav2 launch using the “nav2” command. This will drain the laptop battery.
7. With the system up and running, use the “ml” script in a new terminal to load the desired map and begin Nav2’s AMCL. This is like the “ms” command as it only types out part of the script. You must finish the script with the name and location of the map you are trying to load. Make sure to add “.yaml” to the end so that it looks like the example in **Figure 31** below. Occasionally, this does not work on the first try, so just try again if this is the case. If the map appears on RViz2, you know it has worked. This is shown in **Figure 32**. Notice that the robot is white as no transform for “base_link” to “map” has been published. Another sign that the map has successfully loaded is if the terminal you are using to run RViz2 updates with the white and red message shown in **Figure 33**.

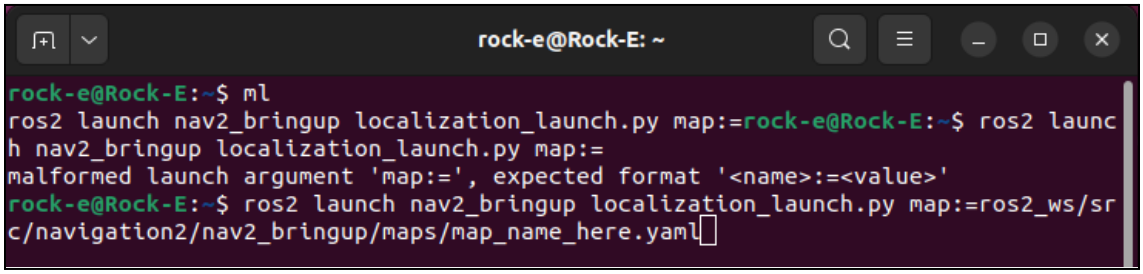


Figure 31: Map Load Example

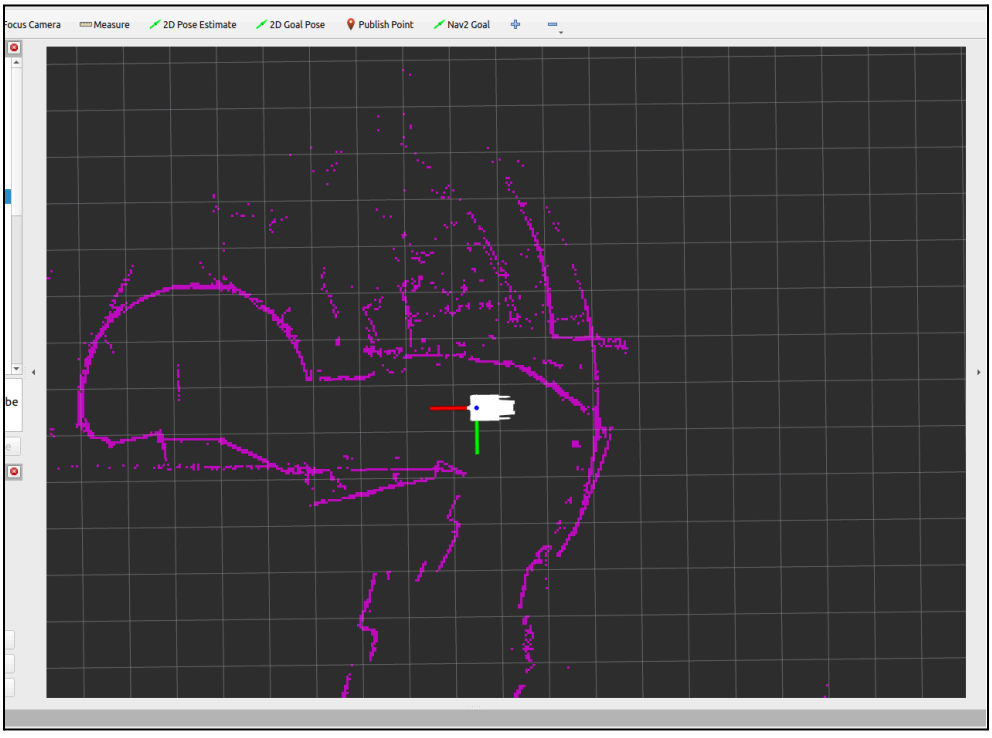


Figure 32: Successful Map Load into RViz2



Figure 33: RViz2 Terminal Output for Successful Map Load

8. With the map loaded, the initial pose of the robot needs to be set. In RViz2, click on the "2D Pose Estimate" button at the top of the window (**Figure 34**). Then, click the approximate spot of the robot within the map with the proper orientation (**Figure 35**). Notice that after it is set, the robot turns gray because a proper transform has been made for the "base_link" into the "map" frame. You can use the LaserScan visualization tool with a subscription to "/scan" in RViz2 to see how the map lines up with the current position of the robot. It is best to set the initial pose multiple times so that the starting point of the robot within the map is accurate.



Figure 34: 2D Pose Estimate Button

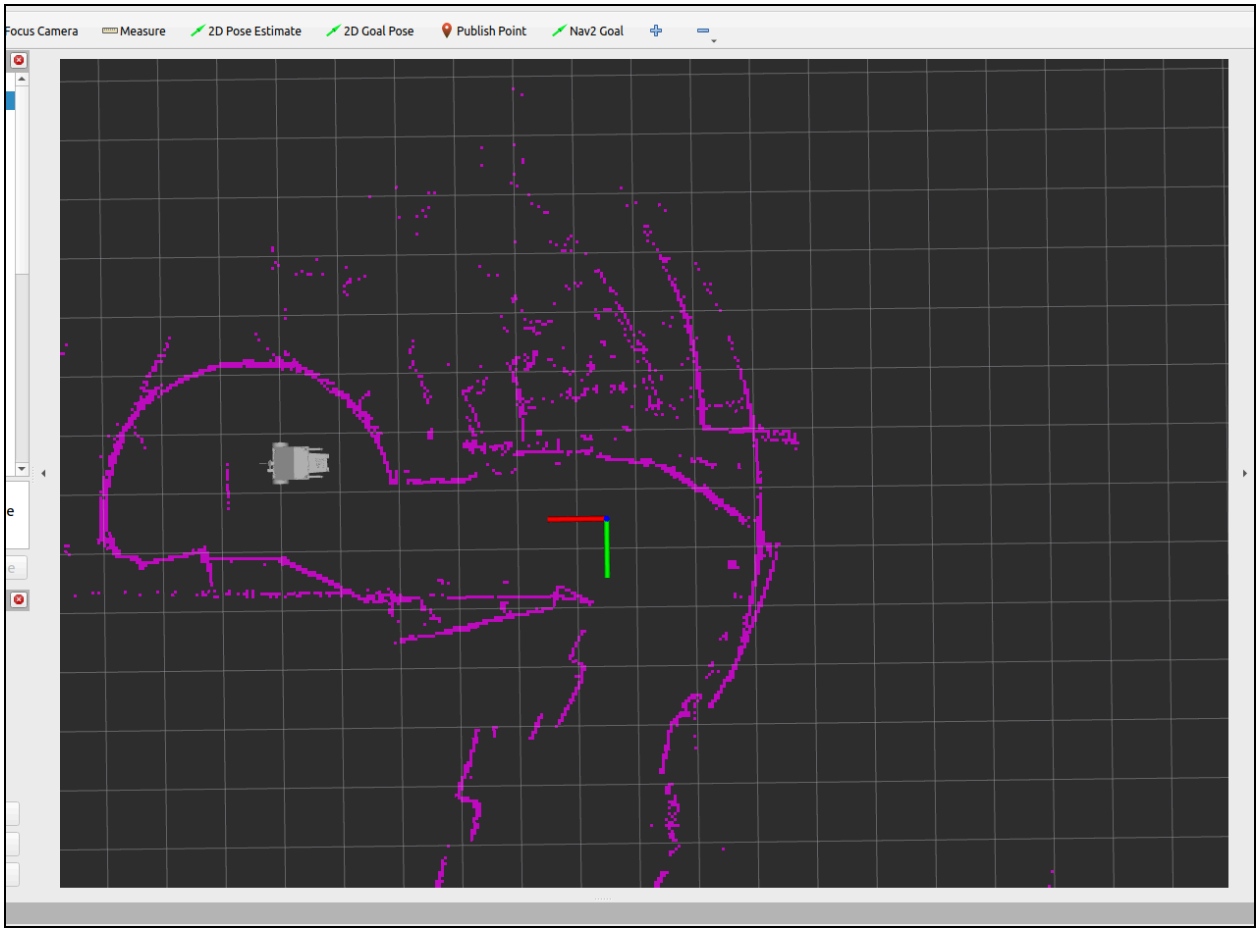


Figure 35: Initial Pose of Robot Within Map

9. After setting the initial pose, the robot is ready to be directed to a waypoint. At the top of the RViz2 window, click on either the "2D Goal Pose" or "Nav2 Goal" Button (**Figure 36**). Much like setting the initial pose, click the desired location and orientation of where you would like the robot to navigate (**Figure 37**). Be careful after releasing the mouse button as the robot will begin navigating.



Figure 36: 2D Goal Pose and Nav2 Goal Buttons

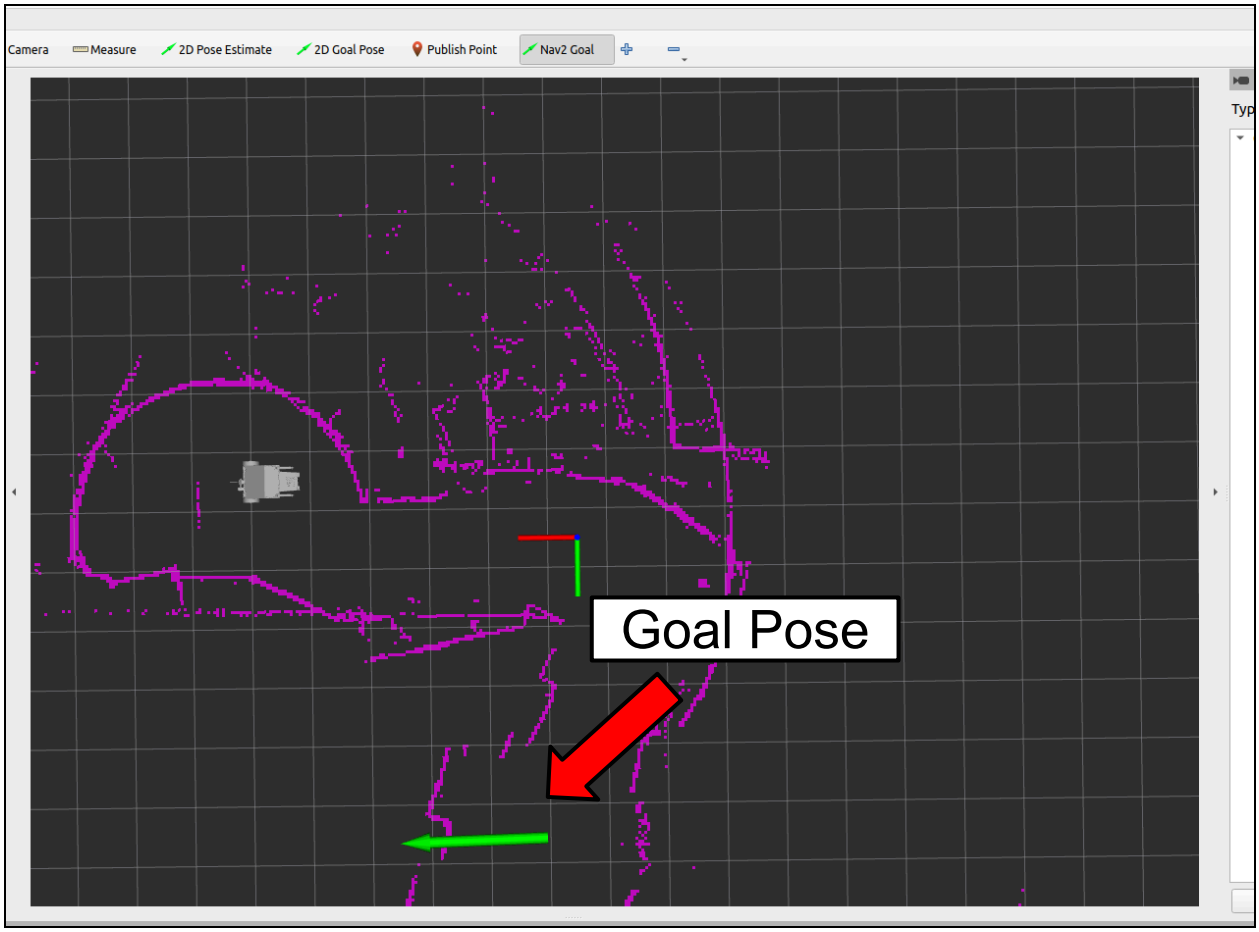


Figure 37: Setting Goal Pose for Map Navigation Mode